Tarea 2: Redis + HBase — del stream a la historia

Procesamiento Masivo de Datos (ELE051-B / EIN102B) · USM 2026-2 · Paralelo 701 · Profesor: Diego Ramírez

Hardware de medición: AMD Ryzen 7 4800H (8C/16T, 2.9–4.2 GHz) · 16 GB RAM DDR4 · Linux (Docker 29.6, Redis 7-alpine, HBase 2.5.15 standalone)

1. Resumen del Pipeline y Cuadratura de Control (Parte B)

El generador con `eventos(t0="2026-11-02T00:00:00-03:00", semilla=2026)` produce **864.000 eventos históricos** (5 días) y **7.200 eventos en vivo** (1 hora a 20× = 25 ms/evento), totalizando **871.200 eventos**. La carga histórica directa por lotes (`batch_size=5000`) insertó los 864.000 eventos en `lecturas` en **32.14 s (26,881 filas/s)** y en `lecturas_tipo` en **32.09 s (26,924 filas/s)** (32.14 s en paralelo). Al finalizar la hora en vivo con los grupos independientes `ingesta` y `tablero` sobre `sensores:t2` (`MAXLEN ~ 10000`), `verificar.py` confirma `XPENDING = 0` en ambos grupos y exactamente **853.953 filas** en cada tabla de HBase (871.200 − 17.243 repetidos exactos − 4 choques de milisegundo por lecturas tardías con distinto valor).

2. Respuestas a las 6 Preguntas (Parte C — Mediana de 5 corridas sin la 1ª)

P1. "Ahora" — Redis contra HBase

Consultas: En Redis, un pipeline con 16 `HGETALL ultimo:<est>` y `ZREVRANGE ranking:flujo 0 4 WITHSCORES` (**1.061 ms**). En HBase, como la clave `estacion#timestamp` ordena ascendentemente, obtener "lo último" exige abrir 16 scanners reversos (`scan(row_prefix=b"<est>#", reverse=True, limit=1)`) y ordenar en Python (**260.988 ms**, es decir, **246.0× más lento**). **Top 5 flujo vehicular:** `SPP-002` (98.0), `HUA-002` (97.1), `HUA-001` (90.1), `PEN-001` (68.6), `CC-002` (62.7).

Estación	Tipo	Redis	HBase	Hora (07-nov)	Estación	Tipo	Redis	HBase	Hora (07-nov)
CC-001	flujo_vehicular	33.2	33.2	00:59:58.000	HUA-001	flujo_vehicular	90.1	90.1	00:59:51.000
CC-002	flujo_vehicular	62.7	62.7	00:59:54.000	HUA-002	flujo_vehicular	97.1	97.1	00:59:59.000
CC-003	ocupacion_estacionamiento	42.6	42.6	00:59:57.500	SPP-001	ocupacion_estacionamiento	66.0	66.0	00:59:56.000
CC-004	calidad_aire	73.4	73.4	00:59:53.000	SPP-002	flujo_vehicular	98.0	98.0	00:59:59.500
CC-005	flujo_vehicular	19.1	19.1	00:59:52.500	CHI-001	calidad_aire	67.8	67.8	00:59:56.500
TAL-001	flujo_vehicular	41.0	41.0	00:59:40.500	PEN-001	flujo_vehicular	68.6	68.6	00:59:47.000
TAL-002	ocupacion_estacionamiento	60.0	60.0	00:59:50.000	COR-001	calidad_aire	19.4	19.4	00:59:43.500
TAL-003	calidad_aire	45.0	45.0	00:59:58.500	COR-002	ocupacion_estacionamiento	62.8	62.8	00:59:55.500

Estación que cambia entre medición y llegada: `TAL-001`. Su última lectura **por medición** es `valor=41.0` medida en `2026-11-07T00:59:40.500-03:00` (ID stream `1794178065223-0`), pero 5 eventos después llegó una **lectura tardía** (última **por llegada**, ID `1794178065348-0`) con `valor=48.9` medida 52,2 s antes (`2026-11-07T00:58:48.335-03:00`). El script Lua del consumidor `tablero` compara `ts_nuevo >= ts_guardado` antes de hacer `HSET`/`ZADD`, impidiendo que la lectura tardía retroceda el tablero.

> **Interpretación P1:** Un dashboard usaría **Redis** (1.061 ms vs. 260.99 ms en HBase, 246.0× más rápido) porque mantiene el último valor y el ranking ya ordenados en RAM, mientras que en HBase con clave `estacion#timestamp` buscar el final de cada prefijo obliga a posicionar 16 scanners en disco/MemStore (salvo que se mantenga una tabla extra con timestamp invertido `10^13 - ms`).

P2. Ventana de 10 minutos — Redis

Consulta: `ZRANGEBYSCORE ventana:<est> (ancla-600000) ancla` anclada a `1794023999500` (`2026-11-07T00:59:59.500-03:00`). Tiempo medido: **2.465 ms**.

Estación	Lecturas (10 min)	Promedio (veh/min)	MEMORY USAGE	Estación	Lecturas (10 min)	Promedio (veh/min)	MEMORY USAGE
CC-001	75	**20.89**	2,616 B	HUA-002	67	**76.50**	2,112 B
CC-002	68	**58.88**	2,104 B	PEN-001	86	**87.34**	3,136 B
CC-005	70	**18.67**	2,616 B	SPP-002	75	**93.13**	2,624 B

HUA-001	62	**92.08**	2,112 B	TAL-001	74	**34.45**	2,624 B

> **¿Por qué ZSET con score=ts y qué pasa sin recortar? (Interpretación P2):** Las lecturas tardías (30–120 s) llegan desordenadas y la frecuencia por estación es aleatoria; el ZSET con `score=ts_ms` y miembro `ts_ms:valor` las ordena por tiempo de medición, deduplica repetidos y recorta exactamente 10 min con `ZREMRANGEBYSCORE`. Con recorte, las 8 ventanas ocupan **19,944 B (19.5 KB)** (`CC-001`: 2,616 B). Sin recortar, `CC-001` sube a **43,907 B** en 1 h y a **5,590,238 B (5.33 MB)** en 5 días (53,656 entradas, ~104.2 B/entrada), por lo que en operación continua crecería linealmente sin cota hasta agotar la RAM.

P3. Un rango en HBase — Scan por rango vs. Tabla entera + filtro

Consulta: Lecturas de `CC-001` el 04-nov en `[08:00, 10:00)`: **891 lecturas**. Scan por rango `[CC-001#2026-11-04T08:00, ...T10:00)`: **17.81 ms**. Scan de tabla entera + `RowFilter`: **4811.77 ms (4.812 s)** → **270.1× más lento**.

Cuarto de hora	Lecturas	Promedio	Cuarto de hora	Lecturas	Promedio
`08:00-08:15`	112	**15.61**	`09:00-09:15`	116	**24.40**
`08:15-08:30`	125	**25.67**	`09:15-09:30`	91	**18.10**
`08:30-08:45`	97	**17.13**	`09:30-09:45`	128	**19.24**
`08:45-09:00`	104	**25.87**	`09:45-10:00`	118	**17.49**

> **Interpretación P3 y extrapolación a 5 años (×365 datos):** Con 5 años en vez de 5 días, el `scan` por rango seguirá tardando **~17.8 ms** (complejidad *O(k)*: salta por el índice de bloques HFile directo al prefijo `CC-001#2026-11-04T08:00`), mientras que el recorrido completo con filtro crecerá linealmente con el tamaño de la tabla (*O(N)* × 365) hasta **~29.3 minutos** por consulta.

P4. Dos patrones de acceso — Redis catálogo + HBase (`lecturas` vs. `lecturas_tipo`)

Consultas (05-nov [07:00, 09:00), calidad del aire): (a) Leer `SMEMBERS tipo:calidad_aire` en Redis + 4 scans por rango en `lecturas` tardó **83.52 ms**; (b) 1 solo scan `[calidad_aire#2026-11-05T07:00, ...T09:00)` en `lecturas_tipo` tardó **58.25 ms**.

Estación	Lecturas (07:00–09:00)	Promedio PM2.5 (µg/m³)	Máximo PM2.5	Hora de la 1ª ocurrencia del máximo
`CC-004`	877	**68.09**	**80.0**	`2026-11-05T07:22:15.000-03:00`
`CHI-001`	904	**62.18**	**80.0**	`2026-11-05T07:07:14.500-03:00`
`COR-001`	890	**22.78**	**52.3**	`2026-11-05T08:18:32.500-03:00`
`TAL-003`	895	**48.14**	**76.5**	`2026-11-05T07:27:43.500-03:00`

> **Costo de la 2ª tabla e interpretación P4:** Mantener `lecturas_tipo` duplica las escrituras en ingesta (×2 `put` por evento) y duplica el espacio en disco (tras `flush`: `123M /data/hbase/data/default/lecturas` · `147M /data/hbase/data/default/lecturas_tipo`). Como HBase no tiene transacciones entre tablas, si falla la 2ª escritura antes del `XACK`, la entrada queda en `XPENDING`; al reintentarse el lote completo, la idempotencia de la row key reescribe ambas tablas sin duplicar filas. **¿Cuándo NO vale la pena?** Cuando el número de estaciones del tipo buscado es pequeño (aquí solo 4 estaciones: hacer 4 scans puntuales en `lecturas` cuesta casi lo mismo que 1 scan en `lecturas_tipo`) o cuando la consulta por tipo es esporádica frente al costo permanente de escribir y almacenar el doble.

P5. Hotspotting y Salting en HBase (Hora en vivo en 3 diseños × 4 regiones)

Diseño de Row Key	Región 1	Región 2	Región 3	Región 4	Total escrituras	Lectura 07-nov [00:10, 00:20) (1180 filas)
1) `estacion#ts`	**2,584** (`(inicio)`)	**1,729** (`COR`)	**1,360** (`PEN`)	**1,375** (`TAL`)	**7,048**	16 rangos: **82.38 ms**
2) `ts#estacion` (día)	**0** (`(inicio)`)	**0** (`11-03`)	**0** (`11-05`)	**7,048** (`11-06`)	**7,048**	1 rango: **22.55 ms**
3) `salt#ts#estacion`	**1,818** (`0`)	**2,219** (`1`)	**1,731** (`2`)	**1,280** (`3`)	**7,048**	4 rangos: **40.77 ms**

> **Interpretación y elección P5:** (Nota: JMX registra 7.048 escrituras por tabla porque `happybase.Batch(batch_size=2000)` agrupa en memoria las claves repetidas dentro de cada lote antes de enviarlas a HBase: 7.047 claves distintas + 1 repetida en borde de lote). `ts#estacion` concentra el **100% de las escrituras (7.048) en la Región 4** (*hotspotting* severo por clave monótona). Elegimos **`estacion#ts`** para este sistema porque distribuye la ingesta entre todas las regiones y resuelve en 1 solo rango las consultas por estación (P3, P4a, P6) con apenas 16 estaciones. **Cambiaríamos a `salt#ts#estacion`** bajo el supuesto de tener miles de estaciones y que la consulta dominante fuera barrer ventanas temporales globales de toda la red (4 rangos fijos paralelizables en vez de miles de scans por estación, sin sufrir el *hotspot* de escritura de `ts#estacion`).

P6. Pregunta propia (Redis + HBase): Anomalía actual de las Top-5 estaciones frente a su historia

Pregunta: *"Para las 5 estaciones con mayor flujo vehicular en este instante (ranking en vivo en Redis), ¿en qué porcentaje y puntaje Z difiere su promedio de los últimos 10 minutos (ventana móvil 00:50–01:00 en Redis) respecto a su media y desviación estándar histórica en esa misma franja horaria (00:50–01:00 de los 5 días previos en HBase)?"* Tiempo medido: **115.97 ms**.

Estación (Top 5 actual)	Último flujo	Promedio 10 min (Redis)	Media ± DE histórica 00:50–01:00 (HBase)	Lecturas hist.	Desviación (Z-score)	Diagnóstico
`SPP-002`	98.0	**93.13**	94.99 ± 12.94	357	**-2.0%** (Z=-0.14)	BAJO_PROMEDIO
`HUA-002`	97.1	**76.50**	69.44 ± 13.34	386	**+10.2%** (Z=+0.53)	SOBRE_PROMEDIO
`HUA-001`	90.1	**92.08**	97.44 ± 11.00	399	**-5.5%** (Z=-0.49)	BAJO_PROMEDIO
`PEN-001`	68.6	**87.34**	72.71 ± 13.15	388	**+20.1%** (Z=+1.11)	SOBRE_PROMEDIO
`CC-002`	62.7	**58.88**	60.67 ± 11.98	376	**-3.0%** (Z=-0.15)	BAJO_PROMEDIO

> **Justificación del reparto P6: Redis decide el *qué* y el *ahora*** (identifica en *O*(log *N*) las 5 estaciones más cargadas con `ZREVRANGE ranking:flujo 0 4` y entrega su ventana reciente `ventana:*` ya recortada en RAM), mientras que **HBase resuelve el *cuándo histórico*** ejecutando solo 5 × 5 = 25 scans acotados `[est#diaT00:50, est#diaT01:00)` sobre los 5 días previos que no viven en la RAM de Redis.

3. Cuando algo se cae — Experimentos de Fallas (Parte D)

D.1 El consumidor `ingesta` muere a mitad de lote (entre `put` y `XACK`)

- **1. Qué quedó en `XPENDING`:** Al forzar `os._exit(1)` en `ingesta-1` justo después del `batch.put()` del lote 5 y antes del `XACK`, `XPENDING sensores:t2 ingesta` reportó **`1` entrada(s) pendiente(s)** a nombre del consumidor **`ingesta-1`** (ID `1794177886298-0`).
- **2. Recuperación por otro consumidor:** Se inició `ingesta-2`, que ejecutó `XAUTOCLAIM sensores:t2 ingesta ingesta-2 0 0-0 COUNT 200`, reclamó la entrada pendiente de `ingesta-1`, la re-escribió en HBase, la confirmó con `XACK` y procesó el resto de la hora en vivo hasta dejar `XPENDING = 0`.
- **3. Cuadratura final en HBase:** Al terminar, `verificar.py` contabilizó exactamente **853.953 filas** en `lecturas` y **853.953 filas** en `lecturas_tipo`: ni faltan ni sobran filas.
- **4. Por qué no duplica filas y qué quedó en la celda:** Al inspeccionar la fila reprocesada `COR-001#2026-11-07T00:00:02.000-03:00` con `cells(row, b'm:valor', versions=3, include_timestamp=True)` se observan **2 versiones** en la misma celda: `[{'valor': '13.1', 'hbase_cell_ts_ms': 1794177886613}, {'valor': '13.1', 'hbase_cell_ts_ms': 1794177886302}]`. Como la row key se construye con la estación y el timestamp de medición (invariantes en el reintento), el segundo `put` cae en la misma fila y solo añade una versión con nuevo timestamp de escritura, devolviendo en `row()` la más reciente sin duplicar la fila.

D.2 Redis se reinicia a mitad de camino (3.000 eventos producidos, 1.500 procesados, 100 en XPENDING)

Configuración de Redis	Momento	Stream (`XLEN`)	Grupos y `XPENDING`	`ultimo:*`	`ventana:*`	`tipo:*` / `ranking`
Sin persistencia (`--save "" --appendonly no`)	Antes → **Después**	3.000 → **0**	2 grupos (pend=100) → **0 grupos**	16 → **0**	8 → **0**	3 / 8 → **0 / 0**
Con AOF + volumen (`--appendonly yes --appendfsync everysec`)	Antes → **Después**	3.000 → **3000**	2 grupos (pend=100, lag=1400/1500) → **Intactos (pend=100)**	16 → **16**	8 → **8**	3 / 8 → **3 / 8**

Ventana de pérdida y elección de diseño: La **ventana de pérdida** corresponde a los eventos que ya entraron a Redis (o están en vuelo desde el productor) pero aún no han sido escritos en HBase ni confirmados con `XACK` (en el experimento: los 1.400 eventos aún no leídos por `ingesta` + los 100 en `XPENDING` = 1.500 eventos si Redis pierde su memoria). En cada momento está **a salvo en HBase** toda la historia ya confirmada con `XACK` (los 864.000 históricos + los 1.500 ya ingestados), pues vive en el volumen de HBase (WAL + HFiles). En un sistema real de monitoreo urbano aceptaríamos una pérdida máxima de **1 segundo** ante un corte eléctrico total del servidor, eligiendo por ello **AOF con `appendfsync everysec`**: resiste al 100% reinicios del contenedor/proceso y solo arriesga ≤ 1 s ante un *kernel panic* o apagón físico, evitando la penalización de latencia de `appendfsync always` (un `fsync` bloqueante en cada `XADD`).

4. Síntesis Arquitectónica, Tiempos, Escala y Lecciones (Parte E)

4.1 Tabla "¿Qué va en cada motor?"

Dato del problema	Motor	Estructura o Row Key	Por qué ahí y no en el otro motor

Stream de ingesta en vivo (última hora)	**Redis**	Stream `sensores:t2` (`MAXLEN ~ 10000`) + Consumer Groups	Desacopla al productor (40 ev/s a 20×) de múltiples lectores independientes (`ingesta` y `tablero`) en RAM con `XPENDING`/`XAUTOCLAIM`; HBase no tiene colas ni grupos de suscripción.
Última lectura de cada estación ("ahora")	**Redis**	Hash `ultimo:<estacion>` (`valor`, `ts`, `tipo`)	Acceso *O(1)* en ~1 ms y actualización condicional atómica vía Lua (`ts_nuevo >= ts_prev`); en HBase requiere 16 reverse scans (246.0× más lento).
Ranking instantáneo (Top 5 flujo)	**Redis**	Sorted Set `ranking:flujo` (score = `valor`)	Redis mantiene el orden en cada inserción (*O*(log *N*)) y entrega el Top 5 directo con `ZREVRANGE`; HBase ordena solo por row key (no por valor cambiante).
Ventana móvil de 10 minutos	**Redis**	Sorted Set `ventana:<estacion>` (score = `ts_ms`, miembro = `ts_ms:valor`)	Ordena lecturas tardías por tiempo de medición y recorta en RAM con `ZREMRANGEBYSCORE` ocupando solo 19.5 KB en total.
Catálogo de estaciones por tipo	**Redis**	Set `tipo:<tipo>` (`SMEMBERS` / `SADD`)	Conjunto pequeño de metadatos (16 estaciones) leído en sub-milisegundos para parametrizar consultas (P4a).
Historia por estación (días a años)	**HBase**	Tabla `lecturas`, row key `estacion_id#timestamp`	Escala a cientos de millones de filas en disco (LSM + HFiles) con scans contiguos por rango de fechas en ~17.8 ms; en Redis 5 años (~28 GB) agotarían la RAM.
Historia por tipo de sensor	**HBase**	Tabla `lecturas_tipo`, row key `tipo#timestamp#estacion_id`	Índice secundario materializado en disco para barrer un tipo de sensor completo en un rango de fechas con un único `scan` (P4b).

4.2 Resumen consolidado de tiempos medidos (AMD Ryzen 7 4800H · 16 GB RAM)

Pregunta	Operación / Patrón comparado	Motor	Mediana (5 corridas)	Comparación / Conclusión
P1	16 últimos valores + Top 5 flujo: Pipeline Redis vs. 16 reverse scans HBase	Redis vs. HBase	**1.061 ms** vs. **260.99 ms**	Redis es **246.0× más rápido** para el dashboard en vivo.
P2	Promedio ventana 10 min en 8 estaciones (`ZRANGEBYSCORE ventana:*`)	Redis	**2.465 ms**	Lectura en ~2,5 ms; recorte mantiene las 8 ventanas en 19.5 KB.
P3	CC-001 (04-nov 08:00–10:00, 891 filas): Scan por rango vs. Scan total + filtro	HBase	**17.81 ms** vs. **4811.77 ms**	El filtro sin row key es **270.1× más lento** (y escalaría a ~29.3 min en 5 años).
P4	PM2.5 05-nov 07:00–09:00: (a) 4 scans en `lecturas` vs. (b) 1 scan en `lecturas_tipo`	Redis + HBase	**83.52 ms** vs. **58.25 ms**	Con 4 estaciones ambos tardan ~lo mismo, pero la 2ª tabla duplica disco y escritura.
P5	07-nov 00:10–00:20 (1180 filas): 16 rangos (`est#ts`) · 1 rango (`ts#est`) · 4 rangos (`salt`)	HBase	**82.38 ms** · **22.55 ms** · **40.77 ms**	`ts#est` lee en 1 rango pero sufre 100% de *hotspot* al escribir (7.048 en 1 región).
P6	Anomalía Top 5 actual (Redis) vs. media/DE histórica 5 días en 00:50–01:00 (HBase)	Redis + HBase	**115.97 ms**	Combina selección/ventana en RAM con 25 micro-rangos históricos en disco.

4.3 Honestidad con la escala y qué no funcionó / qué haríamos distinto

Hasta dónde se extrapolan las mediciones y desde dónde ya no: (1) **Sí se extrapolan** la diferencia de órdenes de magnitud entre un `scan` por prefijo de row key (*O(k)*) y un recorrido completo con filtro (*O(N)*, P3), así como el conteo de escrituras por región en JMX que demuestra el *hotspotting* de `ts#estacion` (P5). (2) **No se extrapolan** los tiempos absolutos de lectura entre 1, 4 y 16 regiones en P4 y P5: en un HBase *standalone* un único proceso RegionServer sobre un mismo SSD NVMe atiende todas las regiones secuencialmente y sin red ni HDFS, por lo que el *hotspotting* no satura un nodo frente a otros ociosos y hacer 4 o 16 scans secuenciales por Thrift local apenas añade unos milisegundos de *round-trip*. En un cluster real con N RegionServers, escribir en una sola región limita el *throughput* a 1/N de la capacidad del cluster, mientras que los 4 rangos del *salting* se ejecutarían en paralelo en 4 servidores distintos.

Qué no funcionó y qué haríamos distinto: (1) Guardar solo `valor` como miembro de `ventana:<estacion>` colisionaba lecturas distintas con igual valor numérico; lo corregimos usando `ts_ms:valor` como miembro. (2) La función nativa `hash()` de Python varía entre procesos por `PYTHONHASHSEED`; usamos `zlib.crc32(estacion) % 4` para que el *salting* de P5 sea 100% determinista. (3) En producción real, ejecutaríamos los 4 o 16 scans de P4a/P5 en paralelo o mediante `MultiRowRangeFilter` en el servidor en vez de llamadas secuenciales por Thrift.